Entrega E2 — Arquitetura do EntregaJá

Base: docs/e1-visao-requisitos.md (RF01–RF12, RNF01–RNF07 e RN01–RN07) e estrutura do modelo E2. Data: 11/11/2026. As metas quantitativas que não constam dos requisitos foram propostas para validação da equipe; não representam resultados medidos.

1. Visão arquitetural e camadas

Estilo adotado

Monólito modular em camadas, com módulos de negócio e contratos internos explícitos. A aplicação começa como uma API Spring Boot única, usando PostgreSQL compartilhado e adaptadores para integrações. Essa opção reduz custo operacional e permite transação local de pedido/estoque; módulos de catálogo e pagamento poderão ser extraídos quando carga, autonomia de implantação ou escala de equipe justificarem a operação distribuída (ADR 0001).

Camadas e responsabilidades

Camada	Responsabilidade	Componentes EntregaJá	Não deve
Apresentação	HTTP, validação sintática, autenticação e DTOs	Controllers de catálogo, carrinho, pedido, pagamento, entrega, avaliação e administração	Acessar SQL/repositórios ou conter regra de negócio
Aplicação	Casos de uso, coordenação entre módulos e limites transacionais	CatalogoService, CarrinhoService, CheckoutService, PagamentoService, EstoqueService, EntregaService, AvaliacaoService	Depender de HTTP ou expor entidade JPA
Domínio	Estado e invariantes do negócio	Produto, Categoria, Carrinho, ItemCarrinho, Pedido, ItemPedido, Pagamento, Estoque, ReservaEstoque, Entrega, Avaliacao, Usuario, Perfil, Endereco, Frete	Depender de framework, gateway externo ou detalhes SQL
Persistência	Consultas e armazenamento	Repositórios Spring Data JPA; PostgreSQL, tabelas por módulo	Decidir transições ou regras
Integração	Traduzir contratos externos e falhas	GatewayPagamentoPort, adaptador HTTP do gateway simulado; cache Redis como adaptador opcional	Vazar protocolo externo para o domínio

Módulos e modelo de domínio

Módulo	Responsabilidade / regras	Entidades e serviços centrais	Requisitos
Catálogo	Produto/categoria, preço, busca paginada; invalida cache ao alterar	Produto, Categoria; CatalogoService, ProdutoRepository	RF01, RF02, RF12
Carrinho	Itens e subtotal; total congela preço e frete vigentes no checkout	Carrinho, ItemCarrinho, Frete; CarrinhoService	RF03, RF04
Pedido	Criação, propriedade do cliente, transições permitidas, histórico imutável	Pedido, ItemPedido, EnderecoEntrega, HistoricoStatus; PedidoService	RF05, RF08, RF09
Estoque	Reserva atômica de todos os itens; baixa na confirmação definida pela regra operacional; nunca saldo negativo	Estoque, ReservaEstoque; EstoqueService	RF06, RF12, RN01
Pagamento	Chave idempotente única por tentativa lógica; timeout, circuit breaker e estado pendente	Pagamento, GatewayPagamentoPort; PagamentoService	RF07, RNF05, RNF07
Entrega	Atribuição e transições de entrega, consulta do histórico	Entrega; EntregaService	RF08, RF09
Avaliação	Uma avaliação autorizada após pedido entregue	Avaliacao; AvaliacaoService	RF10, RN05
Segurança	Identidade, papel e escopo de acesso ao pedido	Usuario, Perfil; SecurityConfig, AuthorizationService	RF11, RN07, RNF06

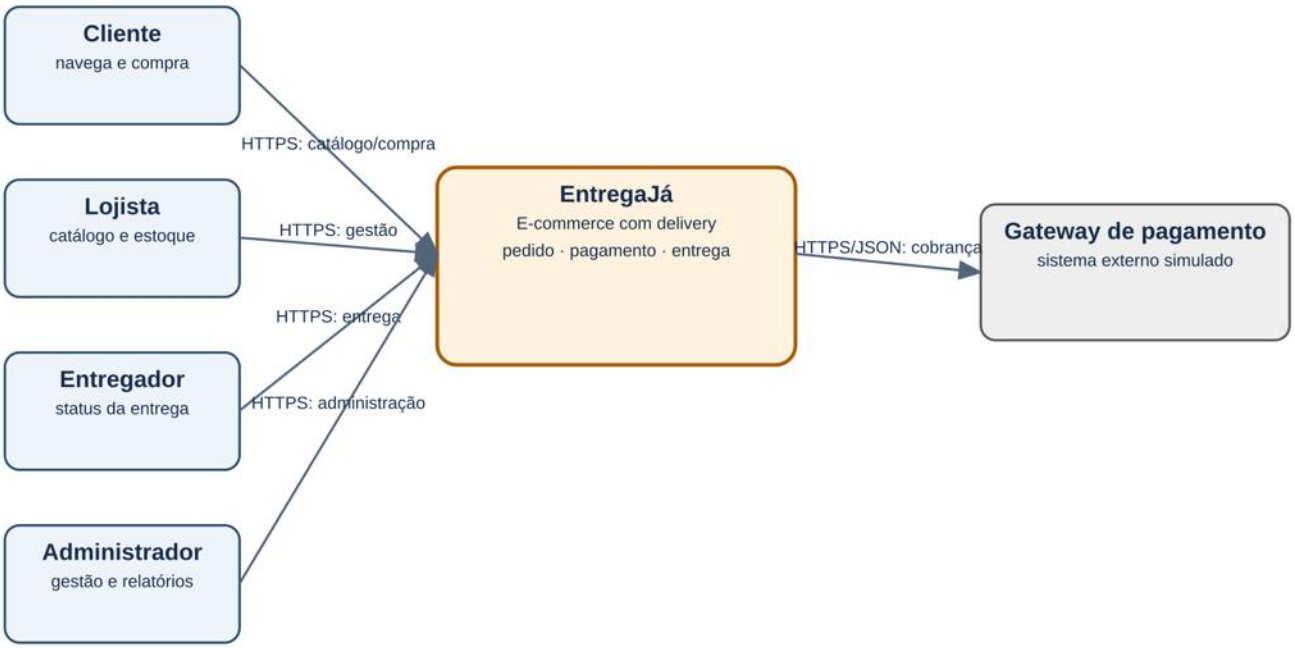
Regras de dependência: dependências de código apontam para o domínio; aplicação depende de portas/interfaces; adaptadores de persistência e integração implementam essas portas. Controllers chamam apenas serviços de aplicação. Módulos colaboram por interfaces públicas de aplicação e não acessam tabelas/repos privados uns dos outros. O checkout coordena Pedido, Estoque e Pagamento; reserva é transacional e o gateway é chamado fora da transação de banco. Preço/frete usados no pedido são snapshots.

Padrões: Repository para persistência; Service/Application Service para casos de uso; DTO para fronteira HTTP; Adapter/Port para gateway; estratégia de cache-aside para leitura de catálogo; chave única e registro persistente para idempotência.

2. Diagramas C4

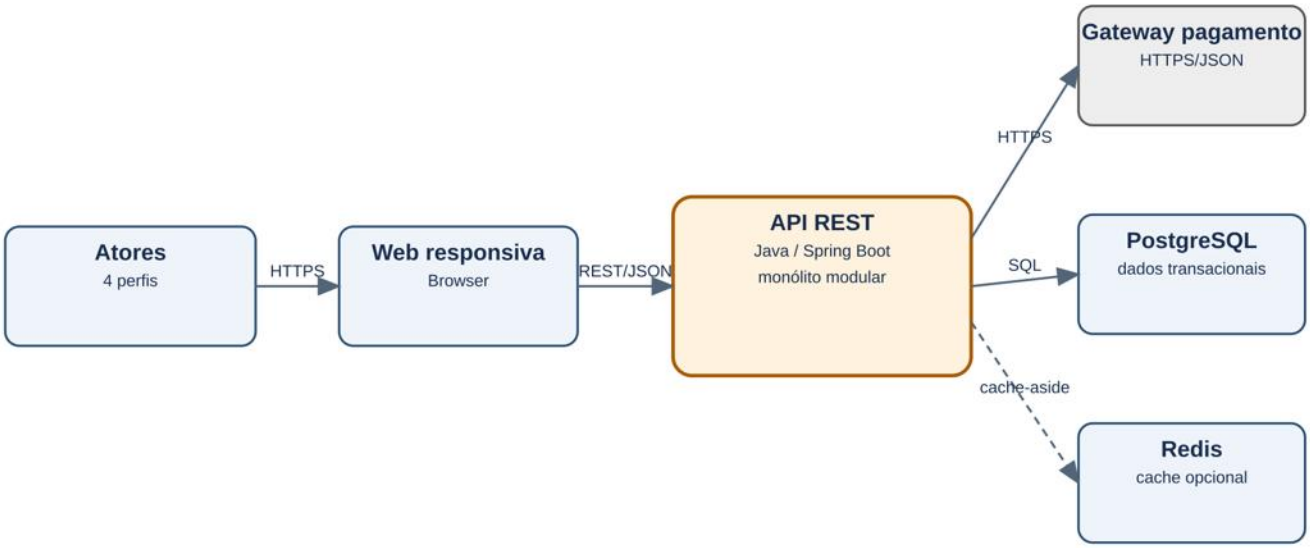
Fontes Mermaid editáveis em docs/arquitetura/fontes/; diagramas SVG em docs/arquitetura/diagramas/. Setas indicam direção da solicitação/dado. HTTP/JSON para API e gateway; SQL para PostgreSQL; cache via protocolo Redis. O diagrama de componentes é um recorte da API, não indica implantação distribuída.

Nível 1 — Contexto



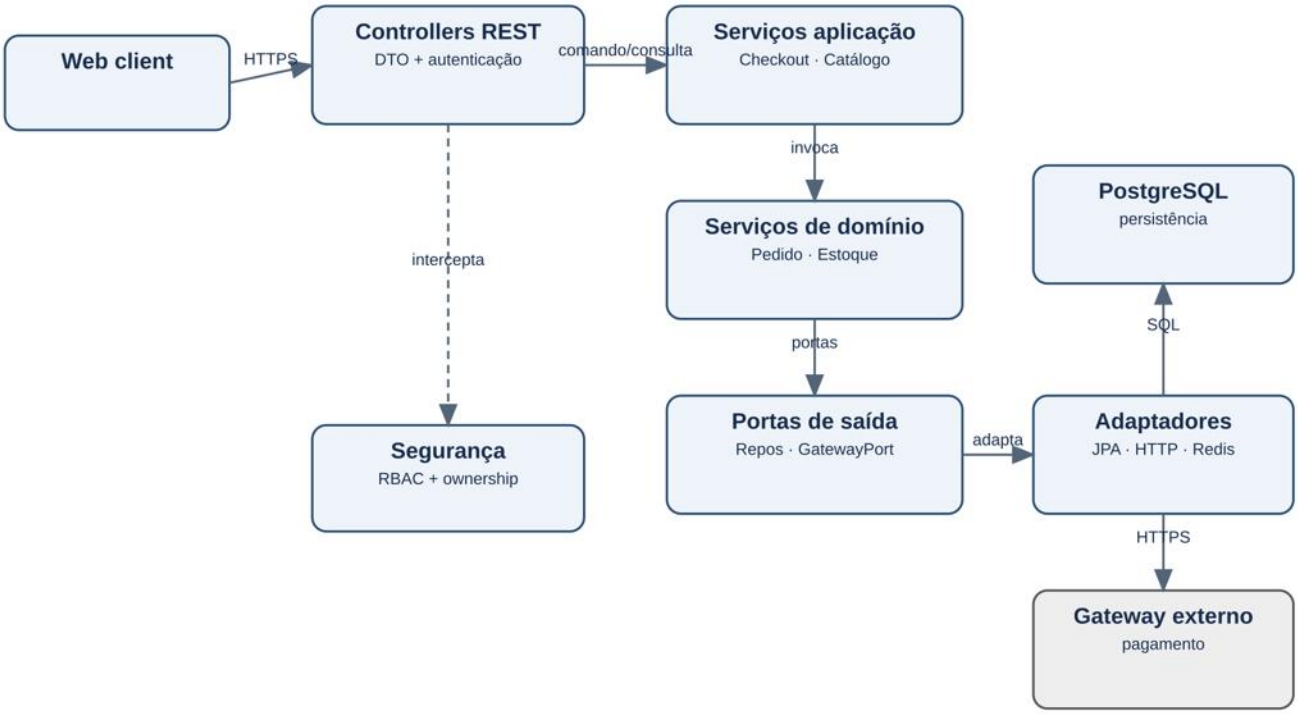
Cliente compra e acompanha pedidos; lojista mantém produtos/preços/estoque; entregador atualiza entrega; administrador gerencia usuários e consulta operação. Todos usam o sistema web pela API HTTPS. O EntregaJá solicita autorização/captura ao gateway externo por HTTPS.

Nível 2 — Contêineres



Cliente web responsivo → API REST Spring Boot → PostgreSQL. A API integra com gateway de pagamento HTTPS. Redis é cache de leitura opcional conforme ADR 0005; sua indisponibilidade resulta em leitura direta no banco. Uma API única reduz custo de operação inicial; separação modular conserva fronteiras para extração futura.

Nível 3 — Componentes



Controllers recebem comandos e consultas; serviços de aplicação coordenam domínios e repositórios. Checkout consulta catálogo, reserva estoque, persiste pedido e delega cobrança ao pagamento; pagamento comunica-se via porta/adaptador. Banco é acessado por repositórios. Segurança é transversal e valida

papel/propriedade.

Modelo de domínio (resumo)

Cliente 1 — N Pedido; Pedido 1 — N ItemPedido; ItemPedido N — 1 Produto (snapshot de descrição/preço); Pedido 1 — 0..1 Pagamento; Pedido 1 — 0..1 Entrega; Produto N — 1 Categoria; Produto 1 — 1 Estoque; Pedido 1 — N HistoricoStatus; Cliente 1 — N Avaliacao. ReservaEstoque associa pedido e produto com quantidade/expiração; restrição atômica garante disponível ≥ 0. Pagamento guarda chave idempotente, referência externa, valor e estado, nunca PAN/CVV.

3. Máquina de estados do Pedido



De → para | Evento/guarda | Efeito

(inicial) → CRIADO | checkout válido | grava itens/preços/frete e pedido

CRIADO → PENDENTE_PAGAMENTO | reserva válida e tentativa iniciada | chave idempotente persistida; gateway chamado com timeout

PENDENTE_PAGAMENTO → PAGO | confirmação do gateway | confirma reserva/baixa; registra histórico

PENDENTE_PAGAMENTO → CANCELADO | recusa definitiva, expiração ou cancelamento explícito | libera reserva; registra motivo

PENDENTE_PAGAMENTO → PENDENTE_PAGAMENTO | timeout/falha transitória | mantém pedido consultável; retry com mesma chave

PAGO → EM_PREPARO | lojista inicia preparo | cria/ativa entrega; só após pagamento (RN02)

EM_PREPARO → EM_ROTA | entregador despacha | registra ator e instante

EM_ROTA → ENTREGUE | confirmação de entrega | encerra entrega; habilita avaliação

CRIADO/PENDENTE_PAGAMENTO/PAGO/EM_PREPARO → CANCELADO | cancelamento explícito autorizado | libera reserva ou inicia estorno conforme estado

Transições não listadas são rejeitadas (RN06). Cancelamento após captura exige estorno rastreável e não apaga o pagamento. Repetição do mesmo evento é idempotente. Estados terminais: ENTREGUE e CANCELADO; disputa/estorno não altera histórico anterior.

4. Cenários de qualidade

Números abaixo são critérios-alvo propostos; confirmar com docente/equipe e validar em ambiente de referência.

ID / requisito | Fonte, estímulo e ambiente | Resposta e medida de sucesso | Tática / verificação

CQ01 RNF01 escalabilidade | 10× tráfego normal durante promoção no catálogo | p95 < 2 s; erro < 1% sustentado por 15 min | cache, paginação, réplicas stateless; carga gradual e monitoramento

CQ02 RNF02 desempenho | Cliente busca nome/categoria em operação normal | p95 < 1,5 s para página de até 20 resultados | índices, paginação, cache; teste de carga com dados representativos

CQ03 RNF03 consistência | 100 clientes concorrentes tentam comprar a última unidade | exatamente 1 reserva/compra confirmada; saldo nunca < 0 | update condicional/lock e transação; teste concorrente e inspeção DB

CQ04 RNF04 disponibilidade | Gateway indisponível por 10 min enquanto catálogo/carrinho são acessados | 99,9% das chamadas de catálogo/carrinho respondem; falha de cobrança não propaga | isolamento, timeout, circuit breaker; injetar falhas e verificar métricas

CQ05 RNF05 resiliência pagamento | Gateway não responde em 2 s ou falha em 5 chamadas consecutivas | abrir circuito; pedido fica PENDENTE_PAGAMENTO; nenhuma duplicidade em 100 retries | chave idempotente persistente, timeout e circuit breaker; teste de falha/replay

CQ06 RNF06 segurança | Usuário tenta acessar pedido de outro cliente ou enviar dados de cartão | 100% dos acessos indevidos negados; nenhum PAN/CVV persistido/logado | HTTPS, RBAC + ownership, tokenização/checkout hospedado; testes de autorização e inspeção de logs/DB

CQ07 RNF07 integrabilidade | Substituir gateway simulado por outro adaptador | núcleo de domínio não muda; até 2 componentes de integração tocados | porta/adaptador; teste de contrato e inspeção de dependências

5. Matriz de rastreabilidade

RNF → decisões, componentes e verificação

Requisito | Cenário | ADR | Componente/tática | Verificação

RNF01 escala 10×, p95 <2s | CQ01 | 0001, 0005 | API stateless, CatalogoService, Redis opcional | carga 10× por 15 min

RNF02 busca p95 <1,5s | CQ02 | 0005 | Catálogo, índice e paginação | teste de carga/telemetria p95

RNF03 estoque sem divergência | CQ03 | 0003, 0004 | Checkout + EstoqueService; reserva transacional e update condicional | concorrência de última unidade

RNF04 catálogo/carrinho sobrevivem à falha gateway | CQ04 | 0001, 0006 | módulo Pagamento isolado, timeout/circuit breaker | injeção de falha e disponibilidade

RNF05 resiliência do pagamento | CQ05 | 0006 | PagamentoService, idempotência e circuit breaker | falha induzida e 100 reenvios sem duplicidade

RNF07 integrabilidade: substituição de gateway | CQ07 | 0006 | GatewayPagamentoPort + adaptadores | contrato e análise de dependência

RNF06 Segurança (dados do cliente e pagamento) | CQ06 | — | SecurityConfig, ownership, tokenização; sem PAN/CVV | testes de autorização e inspeção de armazenamento/logs

RNF07 Integrabilidade (substituição do gateway por adaptador) | CQ07 | 0006 | GatewayPagamentoPort e adaptadores | teste de contrato; núcleo de domínio inalterado

RF → componentes

Requisito | Interface/controller | Serviço | Domínio | Evidência

RF01 manter catálogo/categorias | ProdutoController, CategoriaController | CatalogoService | Produto, Categoria | CRUD e validações

RF02 busca/filtro paginado | ProdutoController | BuscaCatalogoService | Produto, Categoria | paginação, filtro e latência

RF03 carrinho | CarrinhoController | CarrinhoService | Carrinho, ItemCarrinho | recálculo de subtotal

RF04 total/frete | CheckoutController | CheckoutService | Pedido, ItemPedido, Frete | soma e snapshot de checkout

RF05 criar pedido CRIADO | PedidoController | CheckoutService | Pedido, HistoricoStatus | criação e estado inicial

RF06 reservar/baixar estoque | CheckoutController | EstoqueService | Estoque, ReservaEstoque | concorrência e saldo

RF07 pagamento idempotente | PagamentoController | PagamentoService | Pagamento | replay da mesma chave

RF08 mudar estado pedido/entrega | PedidoController, EntregaController | PedidoService, EntregaService | Pedido, Entrega | transições válidas/inválidas

RF09 estado/histórico | PedidoController | ConsultaPedidoService | HistoricoStatus | ordenação e ownership

RF10 avaliar após entrega | AvaliacaoController | AvaliacaoService | Avaliacao, Pedido | bloqueio antes de ENTREGUE

RF11 autenticação/perfis | SecurityFilterChain | AuthorizationService | Usuario, Perfil | matriz de acesso

RF12 reposição estoque | EstoqueController | EstoqueService | Estoque | atualização auditada

Cobertura: todos os RNFs e RFs listados. ADRs 0001→RNF01/04/07; 0002→RF08/RNF04; 0003→RF05/RNF03; 0004→RNF03; 0005→RNF01/02; 0006→RF07/RNF04/05/07. Autenticação e acesso por perfil são cobertos por RF11 e RN07; segurança de dados por RNF06; integrabilidade por RNF07.

6. Tradeoffs

- Consistência × disponibilidade: checkout falha fechado sem reserva consistente; catálogo e carrinho continuam independentes do gateway.

- Síncrono × assíncrono: atualização interna síncrona no monólito simplifica histórico; gateway usa resposta síncrona limitada por timeout, mantendo PENDENTE_PAGAMENTO para confirmação/reconciliação posterior. Eventos são evolução quando houver serviços distribuídos.
- Cache × atualidade: catálogo pode ter TTL curto e invalidação por escrita; checkout sempre valida preço/estoque no dado autoritativo.
- Monólito × microsserviços: menor custo e transações locais agora; aceita escala de implantação menos granular até sinais operacionais justificarem extração.

7. Decisões

Seis ADRs preenchidas no diretório docs/adr/. São decisões de referência arquitetural para validação da equipe, não decisões já aprovadas por integrantes identificados.

Apêndice — Registros de decisão arquitetural

ADR — Monólito modular ou microsserviços para o marketplace

- Status: proposta para validação da equipe
- Data: 2026-11-11
- Responsáveis: equipe EntregaJá (nomes a preencher)
- Requisitos relacionados: RNF01, RNF04, RNF07

Contexto

O produto começa como marketplace de bairro; o catálogo tem pico de leitura, mas checkout precisa de consistência transacional. Operar serviços distribuídos exige implantação, observabilidade e suporte ainda não justificados pelo volume conhecido.

Alternativas consideradas

1. Monólito modular: menor operação e transações locais; escala toda a aplicação e exige disciplina de módulos.
2. Microsserviços desde o início: escala e implantação independentes; alto custo de infraestrutura, rede, observabilidade e consistência distribuída.
3. Monólito sem módulos: simples inicialmente; acoplamento aumenta custo de mudança e extração.

Decisão

Adotar monólito modular em camadas, com módulos e portas explícitas. Reavaliar extração de Catálogo ou Pagamento mediante métricas reais de carga, disponibilidade ou cadência de implantação.

Consequências e custo/benefício

Fronteiras claras e baixo custo inicial; escala vertical/horizontal conjunta e banco compartilhado limitam autonomia. O custo de modularização é baixo frente à complexidade operacional evitada.

Quando rever

Picos sustentados que exijam escalabilidade independente, times autônomos ou falhas de um módulo afetando SLOs.

ADR — Atualização do status do pedido e da entrega

- Status: proposta para validação da equipe
- Data: 2026-11-11
- Responsáveis: equipe EntregaJá (nomes a preencher)
- Requisitos relacionados: RF08, RNF04

Contexto

No monólito, eventos de entrega precisam refletir pedido e histórico. A aplicação inicial prioriza simplicidade; futura distribuição pode exigir propagação assíncrona.

Alternativas consideradas

1. Chamada síncrona entre módulos: consistência imediata e simples de depurar; acoplamento e propagação de falha.
2. Mensageria/eventos: desacopla consumidores e absorve picos; exige broker, outbox, idempotência e consistência eventual.
3. Atualização direta do estado em um único serviço: menor latência, mas mistura responsabilidades.

Decisão

Atualizar pedido, entrega e histórico por chamada síncrona dentro da transação local. Usar eventos de domínio internos; não introduzir broker nesta fase.

Consequências e custo/benefício

Histórico imediatamente consistente e custo operacional menor; futura extração exigirá outbox e consumidor idempotente. Custo/benefício favorece sincronismo no monólito, com fronteira de eventos preparada.

Quando rever

Extração de Entrega/Pedido, atualizações em alto volume, ou necessidade de consumidores independentes e tolerância a indisponibilidade.

ADR — Consistência de estoque: transação imediata ou saga

- Status: proposta para validação da equipe
- Data: 2026-11-11
- Responsáveis: equipe EntregaJá (nomes a preencher)
- Requisitos relacionados: RF05, RNF03

Contexto

Checkout precisa criar pedido e impedir sobre-venda da última unidade. Em uma API modular com banco compartilhado é possível garantir reserva atômica localmente.

Alternativas consideradas

1. Reserva transacional local: forte consistência e fluxo simples; contenção de locks e reserva pode expirar.
2. Saga distribuída: serviços independentes e compensação; consistência eventual, estados intermediários e maior esforço operacional.
3. Verificação sem reserva: barata, mas sofre corrida e vende sem estoque.

Decisão

Reservar todos os itens atomicamente na mesma transação local do checkout, usando update condicional ou lock e chave única da reserva. Confirmar baixa no marco operacional decidido; liberar reserva em recusa/expiração.

Consequências e custo/benefício

Evita saldo negativo e simplifica recuperação; transação/lock limita vazão em produtos disputados. Implementação relacional é barata comparada ao custo de pedidos impossíveis de atender.

Quando rever

Estoque virar serviço independente ou concorrência/latência exigir saga; exigir outbox, compensações e reconciliação antes da mudança.

ADR — Banco por serviço ou compartilhado

- Status: proposta para validação da equipe
- Data: 2026-11-11
- Responsáveis: equipe EntregaJá (nomes a preencher)
- Requisitos relacionados: RNF03, RF05

Contexto

A decisão acompanha o monólito modular (ADR 0001) e reserva transacional (ADR 0003). O caso não informa volume que justifique clusters por módulo.

Alternativas consideradas

1. PostgreSQL compartilhado, tabelas por módulo: transação local e operação simples; risco de consultas/tabelas cruzarem fronteiras.
2. Banco por serviço: isolamento e evolução independente; consistência distribuída e múltiplas operações caras.
3. Banco único sem fronteiras de esquema: barato, porém incentiva dependência acidental.

Decisão

Usar PostgreSQL compartilhado no início, com tabelas e repositórios pertencentes a cada módulo; acesso cruzado só por contrato de aplicação.

Consequências e custo/benefício

Transação de checkout simplificada, backup e monitoramento únicos; contenção e acoplamento físico permanecem. Custo de um banco gerenciado é proporcional ao estágio do produto; limites de módulo reduzem dívida.

Quando rever

Extração de serviço, carga desigual, requisitos legais/isolamento ou contenda comprovada em métricas.

ADR — Cache de catálogo

- Status: proposta para validação da equipe
- Data: 2026-11-11
- Responsáveis: equipe EntregaJá (nomes a preencher)
- Requisitos relacionados: RNF01, RNF02

Contexto

Promoções geram leitura intensa e meta de catálogo p95 <2s sob carga 10×; busca normal p95 <1,5s. Preço e estoque devem ser revalidados no checkout.

Alternativas consideradas

1. Sem cache: leitura sempre atual; banco recebe toda carga e pode saturar.
2. Cache-aside Redis com TTL curto: reduz carga e latência; dado pode ficar brevemente defasado, Redis adiciona operação.
3. Cache indefinido: máximo hit rate; risco inaceitável de preço/disponibilidade antigos.

Decisão

Preparar cache-aside para listagem/detalhe de catálogo com TTL inicial de 60 s e invalidação nas alterações; cache é acelerador opcional. Busca consulta banco/índice quando não for segura para cache. Checkout sempre valida preço/estoque autoritativos.

Consequências e custo/benefício

Reduz carga e ajuda p95; possível defasagem de até TTL e custo de Redis. TTL curto e validação no checkout oferecem benefício de leitura com risco de compra controlado.

Quando rever

Métricas mostrarem baixa taxa de acerto, staleness inaceitável ou custo superior ao ganho; ajustar TTL/invalidação com medição.

ADR — Idempotência e resiliência na integração de pagamento

- Status: proposta para validação da equipe
- Data: 2026-11-11
- Responsáveis: equipe EntregaJá (nomes a preencher)
- Requisitos relacionados: RF07, RNF04, RNF05, RNF07

Contexto

Gateway externo simulado pode falhar ou demorar. Reenvio de uma tentativa não pode cobrar duas vezes; indisponibilidade não deve impedir catálogo/carrinho e o núcleo deve aceitar outro fornecedor.

Alternativas consideradas

1. Chamada direta sem proteção: menor implementação; travamento, duplicidade e acoplamento ao fornecedor.
2. Porta/adaptador + chave idempotente + timeout/circuit breaker: isolamento e reenvio seguro; exige persistir tentativas, política de retry e telemetria.
3. Fila de pagamento assíncrona desde o início: desacopla gateway; infraestrutura e UX de confirmação eventual adicionais.

Decisão

Usar GatewayPagamentoPort com adaptador HTTP. Persistir chave idempotente única por pedido/tentativa; timeout de 2 s, circuit breaker (abrir após 5 falhas consecutivas, janela inicial 30 s), sem retry cego após resultado ambíguo. Manter PENDENTE_PAGAMENTO até confirmação/reconciliação. Nunca armazenar PAN/CVV.

Consequências e custo/benefício

Catálogo/carrinho isolados, troca de gateway localizada, cobrança repetida evitada; status pode demorar e exige reconciliação observável. Esforço moderado de adapter e resiliência compensa o risco financeiro e de indisponibilidade.

Quando rever

Métricas reais pedirem operação assíncrona com fila; fornecedor oferecer idempotência com contrato diferente; mudança em SLO/limites de segurança.